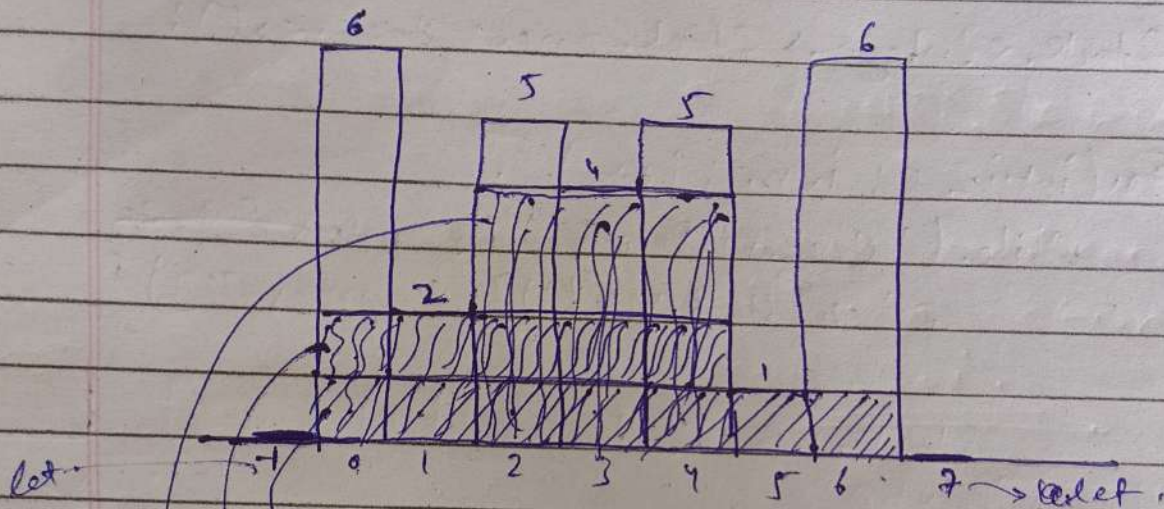


Q5

Maximum Area Histogram

arr [] = 6 2 5 4 5 1 6 $\rightarrow$ Height.



$\rightarrow \text{Area} = 2 \times 1 = 2.$

$\rightarrow \text{Area} = 2 \times 5 = 10.$

$\rightarrow \text{Area} = 4 \times 3 = 12. (\text{max}).$

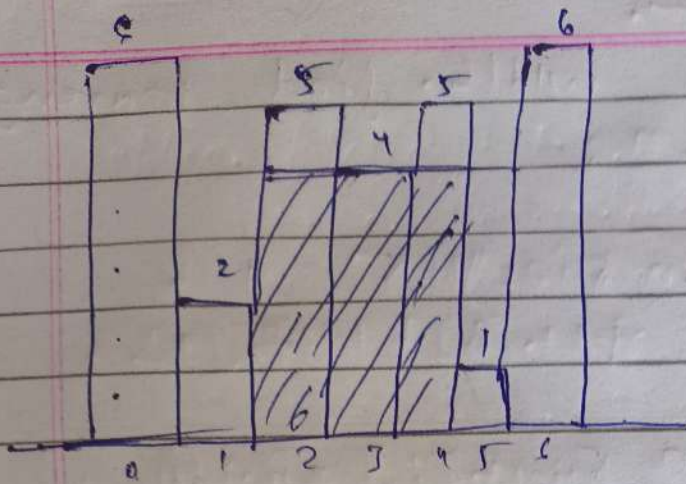
$\rightarrow$ A Building can be expanded into building only when, other $\geq$ current building.

arr [] = 6 2 5 4 5 1 6 $\rightarrow$ is to let kya hai

NSR index $\rightarrow$ right = 1 5 3 5 5 7 7

NSL index left = -1 -1 1 1 3 -1 5

width (right - left - 1) = 1 5 1 3 1 7 1



agr width find krni
hai to element ke left
or right me me ~~smaller~~^{bigger}
element hona chahie.

let hm abh 4 building pe to 4 ke
right me 5 hai aur 4 ke left me bhi
5, lekin hm 5 ke aage nhi ja skte
or nah hi pecheh. biski uske baad
left aur right me 4 se small element
hai.

→ Isiliye hm nearest smallest to right
and nearest smallest to left find
karenge aur jab mil gaye. tabe suru
jayenge. aur width find karenge.

$$\text{ie } \boxed{\text{width} = \text{right}[i] - \text{left}[i] - 1}$$

ex:- let hm 4th building pe hai.

$$\text{width}[] = 5 - 1 - 1$$

$$\boxed{\text{width} = 3}$$

$$\therefore \text{Area} = 4 \times 3 = 12$$

To find area with width find lounge.

Ex

arr : 6 2 5 4 5 1 6
width : 1 5 1 3 1 2 1

Area :-

6	10	5	12	5	2	6
---	----	---	----	---	---	---

Area $[i] = arr[i] * width[i]$

Isme se jo max. hai usko return kar dena hai.

// code:-

class Sol

public int largestRectangleArea (int[] heights) {

int n = height.length;

int[] left = new int[n];

int[] right = new int[n];

Stack<Integer> stack = new Stack<>();

// NSL.

for (int i = 0; i < n; i++) {

while (!stack.isEmpty() && stack.

heights[stack.peek()] >= heights[i]) {

```

        stack.pop();
    }
    if (stack.isEmpty()) {
        left[i] = -1;
    } else {
        left[i] = stack.peek();
    }
    stack.push(i);
}
stack.clear();

```

// NSR

```

for (int i = n-1; i >= 0; i--) {
    while (!stack.isEmpty() &&
           heights[stack.peek()] >= heights[i]) {
        stack.pop();
    }
    if (stack.isEmpty()) {
        right[i] = n;
    } else {
        right[i] = stack.peek();
    }
    stack.push(i);
}

```

~~Code~~

1/ calculated Area:-

```
int maxArea = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
    int width = right[i] - left[i] - 1;
```

```
    int area = height[i] * width;
```

```
    maxArea = Math.max(maxArea, area);
```

```
return maxArea;
```